

Contents

SPECIFICATION — Model Playground (PyQt5 + uv)	1
0. Intent and purpose	2
1. Actors and goals	2
2. Requirements (intent, high level)	3
3. Behavior and state model	4
3.1 System state machine (<code>RunState</code> , per comparison run)	4
3.2 Per-model run state machine (<code>ModelRunState</code>)	4
3.3 Threading model	5
3.4 Streaming vs. non-streaming	5
4. Interfaces / contracts	6
C-01 Core types	6
C-02 <code>Model</code> — the minimal interchangeable interface (R-01)	6
C-03 <code>ModelRegistry</code>	7
C-03b <code>OllamaClient</code> (thin transport for <code>OllamaModel</code> , <code>model.py</code>)	7
C-04 Metrics (<code>metrics.py</code> , pure — testable headless)	7
C-05 Structured output (<code>structured.py</code> , pure — testable headless)	8
C-06 <code>RunWorker(QThread)</code>	9
C-07 Data structures carried to the UI	9
5. UI specification	9
5.1 Top-level layout (<code>MainWindow</code>)	9
C-08 Control widgets and validation	10
5.2 Panels	10
6. Invariants (must hold in every valid implementation)	10
7. Constraints (precise and measurable)	11
8. Edge cases and failure semantics	11
9. Acceptance criteria, tests, and evals	12
9.1 Model interface + mock model (deterministic, no Qt/network)	12
9.2 Metrics (pure) — fast, run always	12
9.3 Structured output (pure) — the reliability boundary	13
9.4 GUI / integration (offscreen, <code>pytest-qt</code>)	13
9.5 Manual / smoke eval (not automated; recorded)	13
9.6 Ollama client (integration, no Qt; network-stubbed)	13
10. Dependencies and environment	14
11. Traceability matrix (id -> where realized)	15

SPECIFICATION — Model Playground (PyQt5 + uv)

- **Status:** v0.1 — draft for implementation
- **Language:** Python 3.12 | GUI: PyQt5 | HTTP: httpx | Schema: jsonschema
- **Curriculum source:** `curriculum/week1/chapter1.md` (§12 Model Playground, §13 Measuring Tokens, §14 Comparing Outputs, §15 Enforcing Structured JSON).
- **Scope of this document:** the *authoritative specification* of an AI-native system. It is written to Level 2–3 (structured, mostly executable): behavior, interfaces, invariants, edge cases, and failure semantics are made explicit so an agent (or engineer) can derive implementation and verification with minimal inference.
- **Principle:** requirements express *intent*; this specification *operationalizes* intent into observable behavior plus the conditions under which we know it is correct.

0. Intent and purpose

Build a small desktop application that treats **different LLMs as interchangeable computational components** sitting behind one common interface, so that an engineer can run the same prompt through several models **side by side** and inspect, per model, not just the text but the *inference-substrate* properties that actually determine application quality:

- **latency** — decomposed into time-to-first-token (TTFT) and total latency;
- **throughput** — generation rate (tokens/second, TPS);
- **economics** — token usage and derived *cost*. Locally there is no vendor bill, so cost is a configurable per-1k-token field for bookkeeping that defaults to 0; the machinery is identical to the hosted case (chapter §13).
- **reliability** — does the output parse, and does it satisfy a schema?

This directly instantiates the chapter’s core thesis: *AI Application = Probabilistic Components + Deterministic Systems*. The model supplies the probabilistic computation (§17); everything here is the **deterministic boundary** — the “harness layer” (§1) and the “reliability boundary” (§15) — that the model is never trusted to provide on its own.

Deployment decision (chapter §3, APIs vs. Local Models): inference is *local*, performed by the **Ollama** runtime (default `http://localhost:11434`) against models already pulled on the machine. The app talks to Ollama’s HTTP API (a small `localhost` HTTP call, not a hosted cloud API) and the model runs on the host’s CPU/GPU/NPU. This is the “local inference” architecture of chapter §3: the app gains control over latency, privacy, availability, version, and configuration, and pays the corresponding engineering burden (owning runtime availability, model download, memory/thermal constraints — see [E-13/E-14/E-15](#)).

Primary product surface: not “chat with a model,” but a **side-by-side evaluation panel** in which, for one prompt, the user sees $M_1(x), M_2(x), \dots$ with each model’s metrics and — optionally — its validated structured output.

Non-goals (explicit, to constrain the solution space):

- No conversation history / multi-turn memory beyond the current prompt (§6 says history is a *context* input, not a feature to build here).
- No tool calling / function-call execution loop (§7 is conceptual; out of scope for this minimal substrate). *See open question Q-04.*
- No multimodal inputs (§8 is conceptual; text-only here).
- No persistence of runs, no cloud/hosted provider, no auth. Ollama is a `localhost` daemon with no key ([E-13](#)). Model **download/pull** is out of scope — the app lists and runs *already-pulled* local models (§3 local-inference model, [E-14](#)).
- No alternative “quality” LLM-as-judge scoring: comparison here is *observational* (metrics + human/JSON inspection), not an automated judge.
- **The app must not require Ollama to import, build, or run its test suite:** a deterministic **MockModel test double** provides every capability for offline/CI use (§9). Ollama is the *real* backend; **MockModel** is what the test suite drives.

1. Actors and goals

Actor	Goals
User (human, single process)	Enter one prompt; pick a set of models + generation parameters; run them in parallel or sequentially; compare outputs, metrics, and costs side by side; optionally obtain and inspect a validated structured output.

Actor	Goals
Model (Model implementation: <code>OllamaModel</code> for the real run, <code>MockModel</code> as a test double)	Given messages + params, produce a (streaming) text completion with usage stats, or fail — never touch the GUI.
Ollama daemon (<i>external</i>)	Local inference runtime on <code>localhost:11434</code> (<code>/api/chat</code> , <code>/api/tags</code>). Owns the model weights and the CPU/GPU/NPU. Not part of this project; E-13/E-14 handle its absence.
Registry (<code>ModelRegistry</code>)	Map a model identifier to a configured model + its pricing, so the rest of the system never branches on provider. For Ollama, populated from <code>GET /api/tags</code> .
Metrics layer (<code>metrics.py</code>)	Turn a run’s raw timings + usage into TTFT / latency / TPS / cost. Pure, headless, testable.
Structured layer (<code>structured.py</code>)	Turn a raw text completion into a <i>validated</i> typed object via <code>parse</code> → <code>validate</code> → <code>accept/reject</code> , with a <code>retry/fallback</code> policy. Pure, headless, testable.
Worker (<code>RunWorker(QThread)</code>)	Drive one model’s generation off the UI thread; emit stream chunks and a final result; crash cleanly.
UI (<code>MainWindow</code>)	Present controls + a grid of per-model result panels; never block on inference.

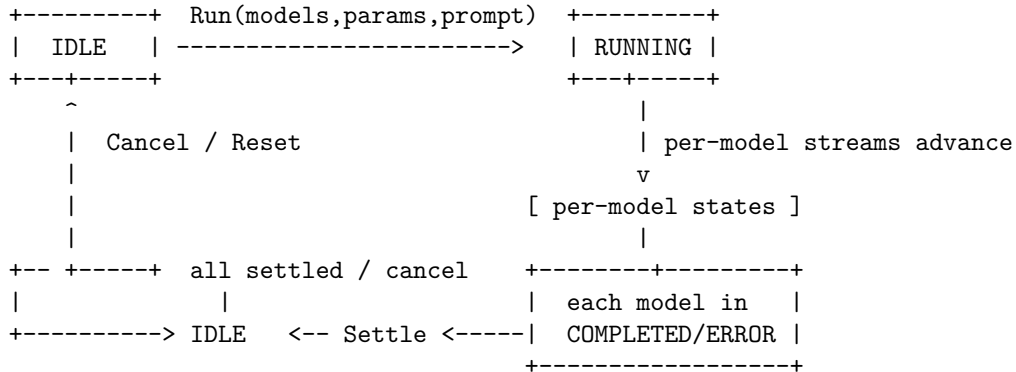
2. Requirements (intent, high level)

ID	Statement
R-01	The application shall expose a common model interface (<code>Model</code>) such that a model can be substituted for another without changing any other layer of the system.
R-02	The user shall specify, per run: a list of one or more models (chosen from the locally-pulled Ollama models, §R-16, or from the built-in <code>MockModel</code> registry when Ollama is unavailable), a generation-parameter set (temperature, <code>top_p</code> , <code>max_tokens</code>), and the prompt (as a message list).
R-03	For each selected model the app shall display the generated text (fully, or streaming token-by-token).
R-04	For each model run the app shall display metrics : <code>tput latency (ms)</code> , <code>ttft_ms</code> , <code>tps</code> , and <code>status</code> .
R-05	For each model run the app shall record token usage : <code>prompt_tokens</code> , <code>completion_tokens</code> , <code>total_tokens</code> . For Ollama these come from the response envelope (<code>prompt_eval_count</code> , <code>eval_count</code>); for <code>MockModel</code> from its deterministic tokenizer.
R-06	For each model run the app shall display a cost computed from usage and that model’s registered per-1k-token prices. For local Ollama prices default to 0 (no vendor bill); the field is configurable so the same machinery models a nominal compute cost (chapter §13).
R-07	The app shall run the selected models side by side , i.e. the user can compare $M_1(x), \dots, M_k(x)$ from one prompt in a single view (§14).
R-08	Streaming mode shall show the response as it arrives and must report $TTFT \leq T_{complete}$ and distinguish them (§12.2, §11).
R-09	When structured output is requested, the app shall emit a validated typed object (not raw prose): { <code>answer</code> , <code>confidence</code> , <code>reasoning_required</code> } (or the configured schema).
R-10	Structured output shall enforce the pipeline raw → parse → validate → accept/reject ; never assume “valid-looking $\geq$ valid” (§6, §15).

ID	Statement
R-11	On a parse/validation failure the app shall apply a retry-then-fallback policy and surface the failure state, never silently fabricate a valid object (§15).
R-12	The GUI must remain responsive while one or more models generate (§3 streaming).
R-13	All model I/O must occur off the Qt main (event-loop) thread.
R-14	The project shall be reproducible via uv on Python 3.12, and runnable/testable fully offline (no Ollama) via MockModel; the real GUI uses Ollama when reachable and degrades gracefully when it is not (E-13).
R-15	A fixed seed + temperature=0 shall make a run deterministic : MockModel is <i>bitwise</i> reproducible; OllamaModel is reproducible <i>best-effort</i> (Ollama honors <code>options.seed</code> , but floating-point/kernel differences across runs may perturb later tokens — token <i>counts and metrics</i> are asserted, bitwise text is not).
R-16	On launch the app shall discover locally-pulled Ollama models via GET <code>/api/tags</code> and populate the model checklist from that list; if Ollama is unavailable it shall fall back to the built-in MockModel registry and state so in the UI (E-13).
R-17	A model that is not pulled locally shall not be runnable: selecting it yields a per-panel ERROR (model not found / pull required) rather than a silent failure or a hard crash (E-14).

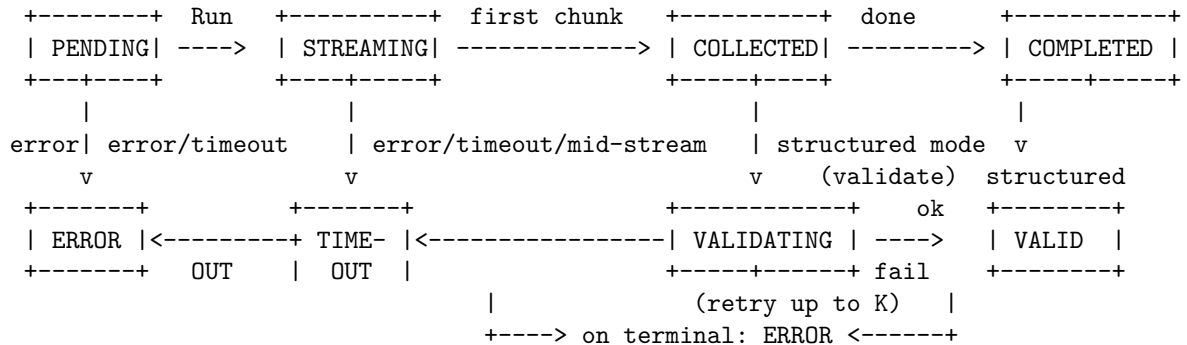
3. Behavior and state model

3.1 System state machine (RunState, per comparison run)



Overall run state is an **aggregate** of the per-model states. A run is settled when *every* model has reached COMPLETED or a terminal error. A single model failure never aborts the others.

3.2 Per-model run state machine (ModelRunState)



State	Meaning	Terminal?
PENDING	Scheduled; request not yet issued.	no
STREAMING	Chunks arriving; <code>text</code> is a partial accumulation.	no
COLLECTED	Full text + usage available (streaming done, or non-streaming returned).	no
VALIDATING	Structured mode: parse + schema-validate the collected text; may retry.	no
COMPLETED	Plain-text mode terminal success (no structured request).	yes
VALID	Structured mode: a validated typed object exists.	yes
ERROR	Terminal fault: request error, timeout, mid-stream failure, or all retries exhausted (§8).	yes
TIMED_OUT	A dedicated terminal when the total elapsed exceeds <code>timeout_s</code> .	yes
CANCELLED	User cancelled the run while this model was active.	yes

Transition rules: - Exactly **one** attempt sequence per model per run; within it, structured mode may perform up to `max_retries` parse attempts (§8). Each retry re-issues generation with an *error-informed* prompt (original prompt + a directive describing the last parse/validation failure). - A model reaching **ERROR**/**TIMED_OUT**/**CANCELLED** is terminal for *that* panel but leaves all sibling panels running. - The overall run reaches **IDLE** (settled) only when **every** panel is terminal. - **Stop/Cancel** from **IDLE** while a run is active tears every model's worker down (no live worker survives; see [I-010](#)).

3.3 Threading model

- **UI thread (main):** widget event loop + all panel updates only.
- **Worker thread(s):** one `RunWorker(QThread)` **per model**, each owning its `Model` and its `generate/stream` call. Never touch widgets.
- Communication is Qt signals only (`QThread` + `pyqtSignal`), plus a per-worker thread-safe cancel flag guarded by a lock. No shared mutable buffers cross threads except via queued (auto-connect) signals.
- **Concurrency policy:** by default all selected models stream **concurrently** (one worker each). A sequential mode (one-at-a-time) is allowed via UI flag to keep `concurrency = 1`; default is **concurrent**.

3.4 Streaming vs. non-streaming

The common interface offers **both**; the UI picks one:

- **Streaming:** the model yields `StreamChunks`; `ttft` is measured to the **first non-done chunk** that carries a non-empty delta. Ollama emits **NDJSON** — one JSON object per line `{"message": {"role","content"}, "done": bool, ...}`; `OllamaModel` splits the response on newlines and parses each line; the **final** line has `done: true` and carries `prompt_eval_count / eval_count` (the [C-01 Usage](#)). `MockModel` yields equivalent `StreamChunks` with no HTTP.
- **Non-streaming:** `OllamaModel` sets `stream=false` and parses the single JSON object; the interface returns one `ModelResponse`; `ttft = total_latency` (a special-case defined explicitly so the metric is always total-ordered, [E-05](#)).
- **Determinism:** generation passes `options.seed` and `temperature`; with `temperature=0` + fixed `seed` Ollama output is reproducible ([R-15](#), best-effort).

4. Interfaces / contracts

C-01 Core types

```
class Role: SYSTEM = "system"; USER = "user"; ASSISTANT = "assistant"

@dataclass
class Message:
    role: str          # one of Role
    content: str

@dataclass
class GenerationParams:    # part of "generation parameters" the model needs
    temperature: float = 0.0 # in [0, 2]; 0 => deterministic-ish
    top_p: float = 1.0 # in (0, 1]
    max_tokens: int = 512 # > 0
    seed: int | None = None # fixed => reproducible when the model supports it (R-15)

@dataclass
class Usage:
    prompt_tokens: int # >= 0
    completion_tokens: int # >= 0
    @property def total_tokens(self) -> int ... # = prompt + completion (I-001)

@dataclass
class StreamChunk:
    delta: str          # text delta; may be "" on a keepalive
    finished: bool      # True on the final chunk
    usage: Usage | None # set (or updated) on the final chunk; None on mid-stream

@dataclass
class ModelResponse:    # the non-streaming return, and the *collected* form
    text: str
    usage: Usage
    model_id: str
```

C-02 Model — the minimal interchangeable interface (R-01)

```
class Model(ABC):
    @property
    def model_id(self) -> str: ... # stable id, e.g. "/ollama/llama3.2", "mock/fast"

    @abstractmethod
    def generate(self, messages: list[Message], **params) -> ModelResponse:
        """Non-streaming. Precondition: messages non-empty.
        Postcondition: return a ModelResponse whose usage.prompt_tokens >= 0.
        May raise on transport/timeout errors (caught by the worker -> ERROR)."""

    @abstractmethod
    def stream(self, messages: list[Message], **params) -> Iterator[StreamChunk]:
        """Streaming. Yields >=1 chunk; the LAST chunk has finished=True and
        carries the final usage (I-008)."""
```

****params** accepts `GenerationParams` fields. **Invariance (I-002)**: no layer other than a concrete `Model` implementation (`OllamaModel`, `MockModel`) may hard-code a provider endpoint, model name, or provider-specific request shape. All other modules see only `Model` + `Message` + `GenerationParams`.

C-03 ModelRegistry

```
@dataclass
class ModelSpec:
    model: Model
    label: str | None = None           # display name; defaults to model.model_id
    price_input_usd_per_1k: float = 0.0
    price_output_usd_per_1k: float = 0.0

class ModelRegistry:
    def __init__(self) -> None ...
    def register(self, spec: ModelSpec) -> None
    def get(self, model_id: str) -> ModelSpec      # KeyError if unknown
    def available(self) -> list[ModelSpec]
    def cost_usd(self, model_id: str, usage: Usage) -> float
```

Invariance (I-003): pricing is bound to the model in the registry, not hard-coded in the UI or the worker. For local Ollama, `price_output_usd_per_1k` and `price_input_usd_per_1k` default to 0.0 (no vendor bill, R-06); the fields remain so a nominal compute cost can be booked in.

C-03b OllamaClient (thin transport for OllamaModel, model.py)

A tiny `httpx`-based client over the Ollama HTTP API. It is the *only* place that knows URLs/JSON shapes (I-002). No import of a vendor SDK; `httpx` is a generic HTTP client.

```
OLLAMA_BASE = "http://localhost:11434"    # overridable via env OLLAMA_HOST
```

```
class OllamaClient:
    def __init__(self, base: str = OLLAMA_BASE, timeout_s: float = 30.0) -> None ...

    def list_models(self) -> list[str]:
        """GET /api/tags -> [name, ...] of *locally-pulled* models (R-16, E-13).
        Raises ConnectionError-like on unreachable daemon; the caller treats that as
        "Ollama unavailable" and falls back to MockModel (E-13)."""

    def chat(self, model: str, messages: list[Message],
             params: GenerationParams, stream: bool) -> "Iterator[StreamChunk] | ModelResponse":
        """POST /api/chat. Maps GenerationParams -> Ollama `options`
        (temperature, top_p, num_predict=max_tokens, seed). Stream: NDJSON, last
        line carries prompt_eval_count/eval_count -> Usage (R-05, I-008).
        Raises on HTTP/network error or malformed NDJSON (E-02/E-07/E-15)."""
```

Mapping (GenerationParams -> Ollama options): `temperature`→`temperature`, `top_p`→`top_p`, `max_tokens`→`num_predict`, `seed`→`seed` (omit when `None`). **Endpoint for unknown model**: Ollama returns HTTP 404/error for a model not pulled locally; `OllamaModel` surfaces that as `KeyError`/panel ERROR (R-17, E-14).

C-04 Metrics (metrics.py, pure — testable headless)

```
@dataclass
class RunMetrics:
    model_id: str
```

```

status: str
ttft_ms: float          # ms to first non-empty delta (or total latency if non-streaming)
total_latency_ms: float
tps: float              # completion_tokens / generation_seconds
usage: Usage
cost_usd: float
retries: int = 0         # structured-mode parse retries that occurred
error: str | None = None

def compute_metrics(
    model_id: str,
    *,
    t_request: float, t_first_token: float | None, t_complete: float,
    usage: Usage, price_input: float, price_output: float,
    retries: int = 0, status: str = "COMPLETED", error: str | None = None,
) -> RunMetrics

```

Definitions (enforced by I-004, I-005, I-006):

$$T_{\text{ttft}} = t_{\text{first token}} - t_{\text{request}} \quad T_{\text{complete}} = t_{\text{complete}} - t_{\text{request}}$$

$$\text{TPS} = \begin{cases} \frac{\text{completion_tokens}}{T_{\text{complete}} - T_{\text{ttft}}} & \text{if } T_{\text{complete}} > T_{\text{ttft}} \\ 0.0 & \text{otherwise (guard, I-005)} \end{cases}$$

$$\text{cost_usd} = \frac{\text{prompt_tokens}}{1000} P_{\text{in}} + \frac{\text{completion_tokens}}{1000} P_{\text{out}} \quad (\text{C-13 unit economics})$$

- **Cost per successful task** (§13) is a *derived* aggregate the UI shows: total cost across a run divided by the count of models that reached a terminal success state (COMPLETED or VALID). If that count is 0, the denominator is 1 and the value is the raw total (never division by zero; I-007).

C-05 Structured output (structured.py, pure — testable headless)

Target schema (the default “answer” schema of §15):

```

{
  "type": "object",
  "additionalProperties": false,
  "required": ["answer", "confidence", "reasoning_required"],
  "properties": {
    "answer": {"type": "string", "minLength": 1},
    "confidence": {"type": "number", "minimum": 0, "maximum": 1},
    "reasoning_required": {"type": "boolean"}
  }
}

@dataclass
class ValidationResult:
    ok: bool
    data: dict | None      # populated iff ok
    errors: list[str]      # human-readable reasons; empty iff ok
    raw: str               # the exact text that was parsed

def parse_json(text: str) -> tuple[dict | None, list[str]]:

```



```

"""Strip a single ```json ...``` fence if present, then json.loads.
Returns (None, [reason]) on any failure - never raises."""

```

```

def validate(data, schema) -> ValidationResult:
    """jsonschema.validate; collect every error message. Never raises."""

```

Pipeline (structured.py):

```

raw text --> strip optional code fence --> json.loads --> jsonschema.validate --> dict
\_____ any step fails => attempt retry (up to max_retries),
           else terminal ERROR with the first failure reason (E-03)

```

C-06 RunWorker(QThread)

```

signals:
    token(model_id: str, delta: str)      # per streamed chunk (UI appends)
    metrics_ready(model_id: str, RunMetrics) # emitted once, on settle or terminal
    crashed(model_id: str, message: str)   # uncaught worker fault (E-07)

public methods (thread-safe to call from main):
    start_run(...) -> None                # kick off the model's generate/stream
    cancel()       -> None                # idempotent; safe from any state

```

Each RunWorker is 1:1 with one model panel. A comparison run owns a QThreadList-style container managing N workers concurrently.

C-07 Data structures carried to the UI

```

ModelPanelView:
    model_id, label, status, text (accumulated),
    metrics: RunMetrics | None,
    structured: ValidationResult | None,
    streaming: bool, done: bool

```

5. UI specification

5.1 Top-level layout (MainWindow)

+-----+ Title: Model Playground - an inference substrate -----+		
LEFT COLUMN (controls)	RIGHT COLUMN: grid of per-model panels	
Prompt (QPlainTextEdit)	+-----+	
system prompt (opt.)	[Mock Fast]	RUNNING
Temperature / top_p /	text: "..." (streaming)	
max_tokens / seed	TTFT __ ms	Lat __ ms TPS _
Models: checklist of	in __/out __ cost \$0.000	
registered models (>=1)	[structured: OK/FAIL badge]	
[stream] checkbox	+-----+	
[structured] checkbox	+-----+	
[sequential] checkbox	[Mock Slow]	COMPLETED
[Run][Cancel]	text: "..."	
state label	TTFT __	Lat __ TPS _ \$__
cost-per-task summary	+-----+	
+-----+		

C-08 Control widgets and validation

Widget	Type	Constraint	Invalid behavior (§8)
Prompt	QPlainTextEdit	non-empty	disable Run; message Prompt must not be empty
Temperature	QDoubleSpinBox	[0, 2]	clamp
top_p	QDoubleSpinBox	(0, 1]	clamp; message if out of range
max_tokens	QSpinBox	1 ... 100,000	message if < 1
seed	QLineEdit(int, optional)	integer or blank	non-integer -> message + disable Run; blank => None
Models	QCheckBox per registered model	≥ 1 selected	disable Run; message Select at least one model
Run / Cancel	QPushButton	enabled per state table (§3.1)	–
stream / structured / sequential	QCheckBox	–	–

5.2 Panels

- Each panel shows: label, status pill, growing text, the four [C-04](#) metrics, usage in/out, cost, and — when structured mode is on — a pass/fail badge plus the parsed JSON or the collected validation errors.
- Streaming: text appends as `token` signals arrive; metrics update once on settle.

6. Invariants (must hold in every valid implementation)

ID	Invariant	Verified by
I-001	<code>Usage.total_tokens == prompt_tokens + completion_tokens</code> ; both ≥ 0.	T-01
I-002	No module outside a concrete <code>Model</code> implementation imports a provider-specific SDK or names a concrete model. (Architecture; checked by T-02 import scan.)	T-02
I-003	Pricing lives only in <code>ModelRegistry</code> ; <code>cost_usd</code> is derived, never hard-coded in UI/worker.	T-03
I-004	<code>ttft_ms ≤ total_latency_ms</code> for every finished run (incl. the non-streaming special case <code>ttft = total</code>).	T-04
I-005	<code>tps ≥ 0</code> ; the zero-generation-duration case yields <code>tps == 0.0</code> (never <code>inf/nan</code>).	T-04
I-006	<code>cost_usd</code> equals the C-04 formula exactly (within float epsilon) for the supplied prices.	T-05
I-007	<code>cost_per_success_task</code> is never a division by zero; all-failed run uses denominator 1.	T-06
I-008	Every <code>stream</code> yields a final chunk with <code>finished=True</code> ; that chunk's <code>usage.completion_tokens</code> equals the streamed token count and equals the non-streaming <code>generate</code> usage for the same input+seed (mock, R-15).	T-07

ID	Invariant	Verified by
I-009	No valid-looking output is accepted: a panel reaches VALID only via <code>validate(...).ok == True</code> . A syntactically valid but out-of-range/missing-field object must NOT validate.	T-08
I-010	At most one live worker per model panel; <code>cancel()</code> leaves zero live workers.	T-09
I-011	All heavy inference is off the UI thread (worker thread id != main thread id).	T-11
I-012	Fixed <code>seed</code> on the mock model yields a bitwise-identical <code>text</code> , <code>usage</code> , and metrics for the same prompt + params.	T-07

7. Constraints (precise and measurable)

ID	Constraint	Measurement
K-01	While RUNNING with all models streaming, a posted task on the event loop is serviced within $p_{95} < 50$ ms.	T-11 (offscreen)
K-02	A mid-stream failure or timeout leaves the panel in a well-defined terminal ERROR/TIMED_OUT ; partial text is preserved for display.	E-02 , T-10
K-03	Default <code>max_retries</code> = 2; default per-run <code>timeout_s</code> = 30.	T-08
K-04	No provider SDK or key is required to import, run, or test the app offline.	T-02 , T-14

8. Edge cases and failure semantics

ID	Situation	Required behavior
E-01	No model selected / empty prompt	Run disabled; inline message; no worker spun.
E-02	A model fails mid-stream (raises inside the iterator, or times out)	Panel -> ERROR/TIMED_OUT ; partial text kept; siblings continue ; the run still settles. Never fabricate completion.
E-03	Structured parse or validation fails	Retry up to <code>max_retries</code> with an error-informed prompt; on exhaustion -> ERROR showing the first failure reason. Never present an unvalidated dict as VALID (I-009).
E-04	Streaming disabled (non-streaming path)	<code>ttft_ms == total_latency_ms</code> ; <code>tps</code> computed over the single interval (guards as in I-005).
E-05	Zero completion tokens (empty response)	<code>tps == 0.0</code> (I-005); not <code>inf/nan</code> ; status may still be COMPLETED .
E-06	<code>max_tokens</code> = 0 or other invalid param	Validation (§5.2) blocks Run; message shown.
E-07	Uncaught worker exception	Emit <code>crashed(model_id, msg)</code> ; panel -> ERROR ; never abort the whole process.

ID	Situation	Required behavior
E-08	Cancel while active	Every worker torn down and joined; no live worker survives (I-010); clean exit.
E-09	Unknown model id requested	<code>ModelRegistry.get</code> raises <code>KeyError</code> -> surfaced as panel <code>ERROR</code> , not a crash.
E-10	No display (CI)	Tests use <code>QT_QPA_PLATFORM=offscreen</code> ; logic unaffected.
E-11	Code-fence-wrapped JSON	<code>parse_json</code> strips a single json ... fence before loads; bare JSON also accepted.
E-12	Duplicate concurrent runs from the UI	Cancel any prior live run first; exactly one active run at a time (I-010).
E-13	Ollama daemon unreachable (not running / <code>OLLAMA_HOST</code> wrong)	<code>OllamaClient.list_models()</code> raises; on launch the UI falls back to the MockModel registry with a status banner <code>Ollama unavailable - using mock models</code> (R-16); no crash, no hang.
E-14	Model not pulled locally (Ollama returns <code>HTTP 404/model not found</code>)	That panel -> <code>ERROR</code> with <code>model not found: 'foo'</code> - pull it with <code>ollama pull</code> . Siblings continue; no crash (R-17 / E-09).
E-15	Malformed NDJSON line / missing <code>eval_count</code> in an Ollama stream	Skip non-JSON keepalive lines; on a malformed final line, keep partial text and set <code>usage</code> to best-effort <code>count_tokens</code> on the accumulated text (or 0) rather than crash; surface a warning on the panel (E-02).

Failure philosophy (spec-engineering doctrine): the model is the *probabilistic* side of the reliability boundary (§15); everything in §4 [C-04](#)/[C-05](#) is the *deterministic* side. The dominant failure here is **accepting an unvalidated artifact as valid** and **letting one model's failure poison the others**. This spec forbids both: [I-009](#) enforces the validate gate; [E-02](#)/[E-07](#) isolate faults per panel.

9. Acceptance criteria, tests, and evals

All tests target Level-3 executable criteria. Pure layers ([C-04](#), [C-05](#)) and mock models need **no Qt and no network**; GUI tests run offscreen via `pytest-qt`.

9.1 Model interface + mock model (deterministic, no Qt/network)

ID	Criterion
T-01	Usage arithmetic (I-001): <code>Usage(p,c).total_tokens == p + c</code> for representative pairs.
T-02	Interface purity (I-002 / K-04): a source scan asserts the <i>only</i> provider-aware module is <code>OllamaClient</code> ; no module imports a vendor SDK. <code>httpx</code> is allowed only inside <code>OllamaClient</code> . The app imports + runs with no Ollama and no keys (via <code>MockModel</code>).
T-07	Reproducibility (I-012 / I-008 / R-15): <code>MockModel</code> with <code>seed=42</code> yields bitwise-identical <code>text</code> and <code>usage</code> across two runs; its streamed token count equals its non-streaming <code>usage.completion_tokens</code> . Determinism of TTFT/latency is not asserted (wall-clock), but token counts are.

9.2 Metrics (pure) — fast, run always

ID	Criterion
T-03	Pricing location (I-003): changing a model's registry price changes <code>cost_usd</code> in the UI/worker with no other edit.
T-04	TTFT ordering & TPS guard (I-004, I-005): <code>ttft_ms <= total_latency_ms</code> ; <code>zero-generation-duration ==> tps == 0.0</code> (not <code>inf/nan</code>).
T-05	Cost formula (I-006): <code>cost_usd</code> matches $N_{in}P_{in} + N_{out}P_{out}$ exactly for non-trivial prices.
T-06	Cost-per-task (I-007): all-failed run divides by 1; mixed run divides by the success count.

9.3 Structured output (pure) — the reliability boundary

ID	Criterion
T-08	Validate gate (I-009 / R-10): (a) a conforming object validates ok; (b) an out-of-range confidence (e.g. 1.5) fails ; (c) a missing required field fails ; (d) extra property fails (<code>additionalProperties:false</code>); (e) after <code>max_retries</code> exhaustion the panel is ERROR , never VALID .
T-14	Fence + bare JSON (E-11): <code>parse_json</code> accepts both <code>''\n{...}\n''</code> and bare <code>{...}</code> ; both route to the same schema; a non-JSON string returns (<code>None</code> , <code>reason</code>).

9.4 GUI / integration (offscreen, pytest-qt)

ID	Criterion
T-09	Single worker / cancel (I-010 / E-12): starting a run while one is active cancels the prior; after <code>cancel()</code> zero workers are alive.
T-10	Isolated failure (E-02 / K-02): with two mock models, one forced to raise mid-stream, that panel is ERROR while the sibling is COMPLETED ; the run settles.
T-11	Off-thread + liveness (I-011 / K-01): while a run streams offscreen, a posted task is serviced < 50 ms; the worker thread id $\neq$ the main thread id.
T-13	State machine (§3.1/§3.2): idle $\rightarrow$ run $\rightarrow$ (all settle) $\rightarrow$ idle; cancel from running reaches idle with terminal panels; control-enable flags match §3.1.
T-15	Validation UI (E-01): empty prompt and zero models selected disable Run with the inline messages.

9.5 Manual / smoke eval (not automated; recorded)

- Launch with `uv run model-playground`; confirm 2–3 models side by side: one **local Ollama** model (e.g. `mock/fast`, or a real `/api/tags` model if Ollama is running), one slow, one misbehaving (raises), and observe independent panels, streaming text, metric columns, and a structured pass/fail badge (E-02, qualitative).
- Confirm the Ollama-unreachable banner: with no daemon, the checklist falls back to the `MockModel` registry (E-13/R-16).

9.6 Ollama client (integration, no Qt; network-stubbed)

ID	Criterion
T-16	Discovery + fallback (R-16 / E-13): with <code>OllamaClient</code> pointed at an unreachable host, <code>list_models()</code> raises a connection error and the app falls back to <code>MockModel</code> — asserted by a <code>conftest</code> fixture that forces <code>OLLAMA_HOST</code> to a dead port; no network is real.
T-17	NDJSON + param mapping (C-03b / I-008 / E-15): using <code>httpx.MockTransport</code> (no real network), <code>OllamaClient.chat</code> (a) maps <code>max_tokens</code> → <code>num_predict</code> , <code>seed</code> → <code>seed</code> , <code>temperature/top_p</code> through; (b) parses a multi-line NDJSON stream into <code>StreamChunks</code> whose final chunk carries <code>finished=True</code> and <code>usage</code> from <code>eval_count/prompt_eval_count</code> ; (c) keeps partial text and best-effort usage when a trailing line is malformed.
T-18	Unknown model (R-17 / E-14): a mock 404/model not found response yields panel <code>ERROR</code> with the <code>model not found: '...'</code> message; it does not crash.

10. Dependencies and environment

Concern	Decision	Rationale
Package/env manager	uv	Fast, reproducible, pins Python; satisfies R-14.
Python	3.12 ($\geq 3.12, < 3.13$)	Requested; stable PyQt5.
GUI	PyQt5 (5.15)	Requested
HTTP	httpx (stream-capable)	Thin client for <code>OllamaModel</code> (C-03b) over the Ollama localhost API; not imported by pure layers (metrics/structured) nor by <code>MockModel</code> ; not required to run tests (K-04).
Local inference engine	Ollama (<i>external</i>)	Runtime at <code>http://localhost:11434</code> (§0 deployment decision). Not a Python dependency — a host prerequisite. The app degrades to <code>MockModel</code> when it is absent (E-13).
Schema validation	jsonschema	C-05 validate.
Mock model	in-repo MockModel	Deterministic, offline, reproducible (R-15); the default test/runtime double; provides streaming + a “slow” and a “raising” variant for E-02/E-07.
Dev deps	pytest, pytest-qt, ruff	Automated Level-3 tests (§9) + lint.
GUI test backend	QT_QPA_PLATFORM=offscreen	Headless CI (E-10).

Reproducibility (see `README.md`):

```
# host prerequisite for the real path (optional; the mock path needs neither):
#  ollama pull llama3.2
uv sync                                # create .venv (Python 3.12), install everything
uv run pytest                          # run the §9 suite (fully offline; no Ollama needed)
uv run model-playground                # launch GUI (Ollama if reachable, else mock models)
```

11. Traceability matrix (id -> where realized)

Spec id / requirement	Where realized (component / module)	Verified by (tests / evidence)
R-01	C-02(Model), C-03(Registry)	T-02
R-02	C-01(GenerationParams), C-08 controls	T-15
R-03	C-06(token signal), C-07 panel	T-11
R-04	C-04(RunMetrics)	T-04
R-05	C-01(Usage), I-001	T-01
R-06	C-03(cost_usd), I-003/006	T-03, T-05
R-07	§5.1 grid, C-06 N workers	T-10, T-13
R-08	C-04(ttft/tps), §3.4	T-04
R-09/10	C-05(parse,validate), I-009	T-08
R-11	§8 E-03, C-05 retry	T-08
R-14	§10, pyproject, K-04	T-02
R-15	C-01(seed), MockModel	T-07
I-007 / §13	C-04 cost-per-task	T-06
I-010 / E-12	C-06(cancel), §3.2	T-09
I-011 / K-01	§3.3 worker threads	T-11
E-02 / K-02	§3.2 per-model terminal	T-10
R-05 / R-15	C-01(Usage: Ollama eval_count / prompt_eval_count), I-001/008/012	T-01, T-07
R-16 / E-13	C-03b(list_models), C-03 discovery, §8	(smoke, E-13)
R-17 / E-14	C-03b(chat 404 -> ERROR)	T-10-style
C-03b / I-002	OllamaClient (only provider-aware module)	T-02
C-03 / I-003	local default price 0.0	T-05
E-15	C-03b(NDJSON parse: keep partial + best-effort usage)	(smoke)

Open questions / ambiguities flagged for the human (spec elicitation):

1. **Structured schema.** The default schema is the §15 “answer” object. Should it be *user-editable* in the UI, or fixed to this one object for v0.1? (spec: fixed for v0.1; Q-01).
2. **Pricing source.** Prices are hand-entered per model in the registry for v0.1. Should we pull a canonical price table? (Q-02 — recommend: hand-entered, documented as illustrative.)
3. **Default local model.** Which Ollama model(s) should the GUI pre-select for the first smoke run? (Q-03 - recommend: pre-select the first model returned by `/api/tags`, plus the built-in `mock/fast` and `mock/slow`.)
4. **Tool calling.** §7 (tool calling) is deliberately **out of scope** here, as is §8 (multimodal). Confirm these remain conceptual-only in v0.1. (Q-04)
5. **Concurrency default.** Concurrent (one worker/model) vs. sequential default. Spec picks **concurrent**, with a sequential checkbox. (Q-05 — confirm.)
6. **Retry semantics for non-structured mode.** Retries apply only to the structured pipeline; plain-text mode has no retry. Confirm. (Q-06)

End of specification. This document is the source of truth; implementation and tests are to be derived from it and kept in sync per §11.